Delta Robot — Blind Pick-and-Place

How It Works: ROS 2 Node & Simulink / Stateflow Model

`blind_pick_place.py` · `delta_blind_pick_place_sim.m`

1 Overview

Blind pick-and-place moves a delta robot from a known **pick position** to a known **place position** without any camera feedback. The end-effector path is planned entirely in software using a **trapezoidal velocity profile**, and the gripper is a pneumatic solenoid valve commanded over CAN bus.

The same logic exists in two forms. The **ROS 2 node** (`blind_pick_place.py`) runs on the physical robot. The **Simulink / Stateflow model** (`delta_blind_pick_place.slx`) simulates the identical sequence entirely in MATLAB — same IK, same trajectory planner, same FSM states — so you can verify timing, tune speed presets, and inspect motor angles before touching hardware.

2 System Architecture

2.1 ROS 2 node

The node `BlindPickAndPlace` owns three collaborating objects:

Component	Class	Role
Motor controller	DeltaMotorController	3 × RS-00 motors via CAN (can0, IDs 1–3)
Pneumatic gripper	PneumaticGripper	Solenoid valve via CAN (ID 0x04)
Pick sequence	Thread (FSM)	Non-blocking pick-and-place state machine

2.2 Simulink model

The `.slx` model generated by `delta_blind_pick_place_sim.m` contains:

Block	Type	Purpose
BlindPickPlace_FSM	Stateflow Chart	Full FSM — all 6 states, IK calls, trajectory stepping
Trigger	Pulse Generator	Simulates /delta/trigger_pick service call
pick_x/y/z	Constant ×3	Pick position in mm (default: 0, 80, –380)
place_x/y/z	Constant ×3	Place position in mm (default: 80, 0, –380)
SimTime	Clock	Simulation time → gripper settle timer
AngleMux + Scopes	Mux + Scope ×3	Display state_id, θ■/θ■/θ■ (rad), gripper

3 Finite State Machine

The pick-and-place sequence is a six-state Moore machine. One cycle executes per trigger event. While a cycle is running the **busy** flag blocks re-triggering. An IK failure in any motion state causes an immediate jump to RESETTING and opens the gripper as a safety action.

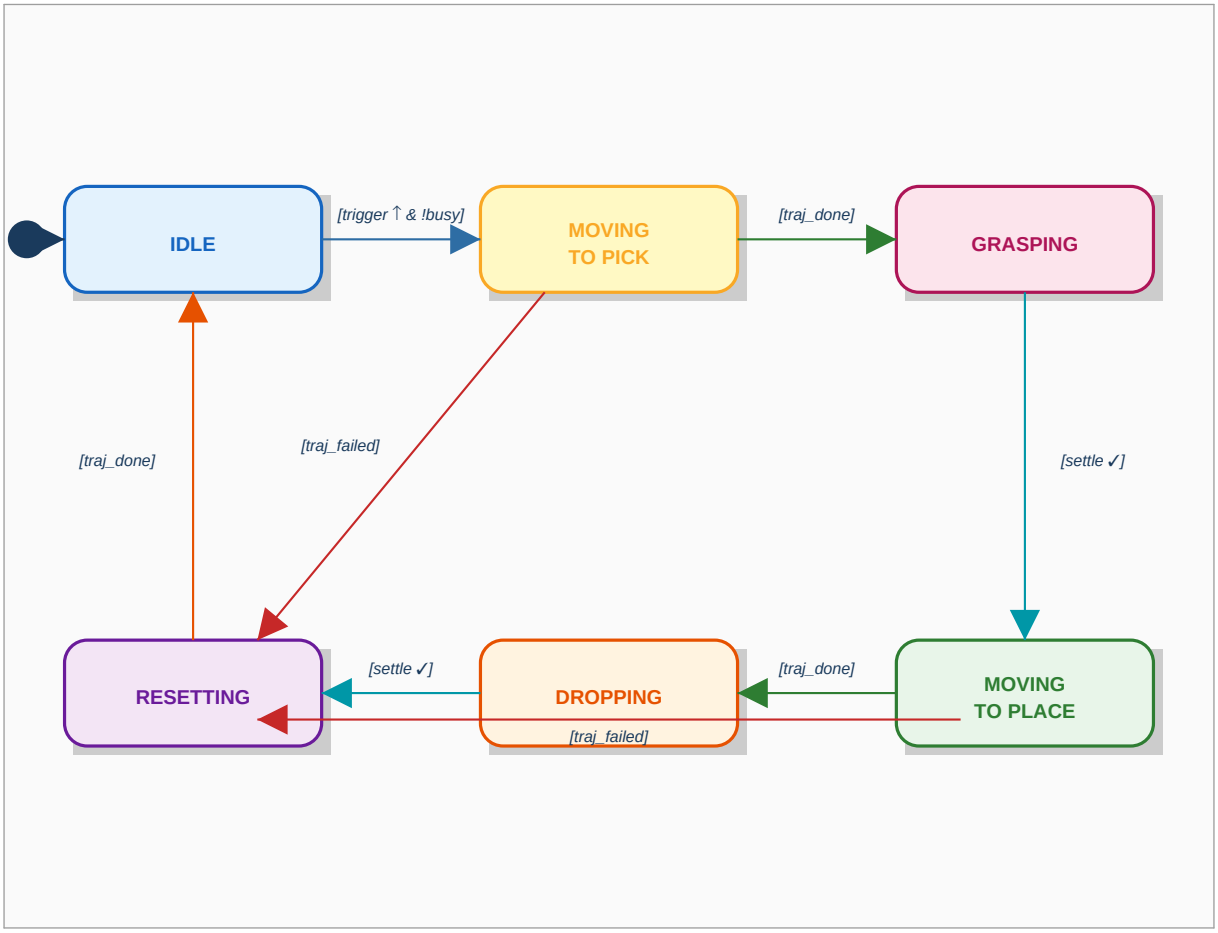


Figure 1 — Blind pick-and-place FSM. Blue = normal flow, orange/red = error paths.

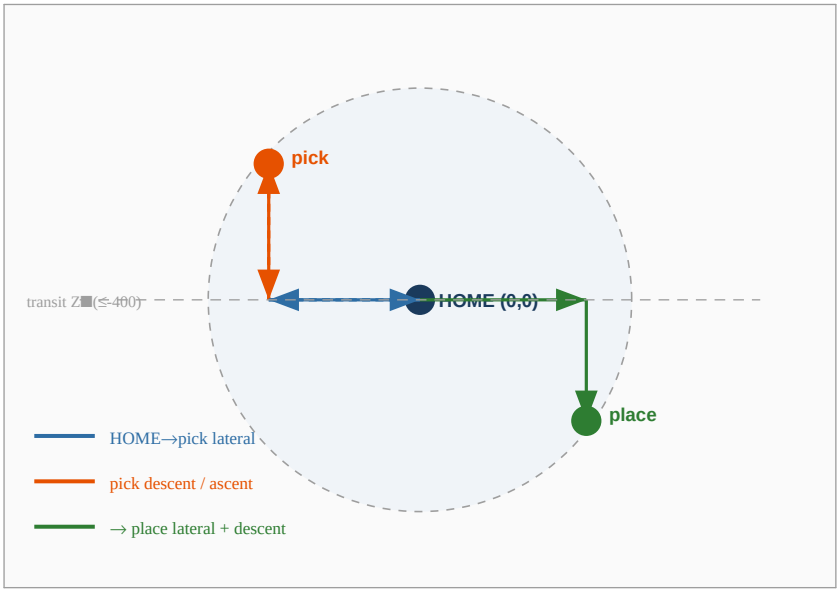
State	state_id	Entry action	Exit condition
IDLE	0	busy=0, gripper open, reset flags	trigger rising edge & !busy
MOVING_TO_PICK	1	Build 3-seg traj HOME→pick, wp_idx=1	traj_done (all waypoints sent)
GRASPING	2	gripper_out=1, start 0.5 s timer	sim_t ≥ settle_timer
MOVING_TO_PLACE	3	Build 4-seg traj pick→place, wp_idx=1	traj_done
DROPPING	4	gripper_out=0, start 0.5 s timer	sim_t ≥ settle_timer
RESETTING	5	gripper_out=0, build 3-seg traj→HOME	traj_done

4 Transit-Z Waypoint Routing

The delta robot workspace is **non-convex near the ceiling**: a straight diagonal from pick to place can exit the reachable sphere. The solution — used identically in the ROS 2 node and the Simulink model — is to route every motion through the **centre column (X=0, Y=0)** at a **safe transit depth**.

Variable	Value	Meaning
SAFE_TRANSIT_Z	–400 mm	Z where workspace radius ≥ 200 mm in all directions

transit_z	min(pick_z, place_z, –400)	Chosen per cycle so both endpoints are above it
home_t	(0, 0, transit_z)	Centre column at transit depth
pick_t	(pick_x, pick_y, transit_z)	Directly above pick point
place_t	(place_x, place_y, transit_z)	Directly above place point



MOVING_TO_PICK (3 segments)	HOME →
MOVING_TO_PLACE (4 segments)	pick_xyz
RESETTING (3 segments)	last_pos

Figure 2 — XY projection of the transit-Z path (left). Path breakdown per motion state (right).

5 Trapezoidal Trajectory Planning

Each straight-line segment is sampled with a **symmetric trapezoidal velocity profile** at a fixed time step (*dt* = 0.05 s, 20 Hz). If the segment is too short to reach *v_max*, the profile degrades gracefully to a triangle (no cruise phase).

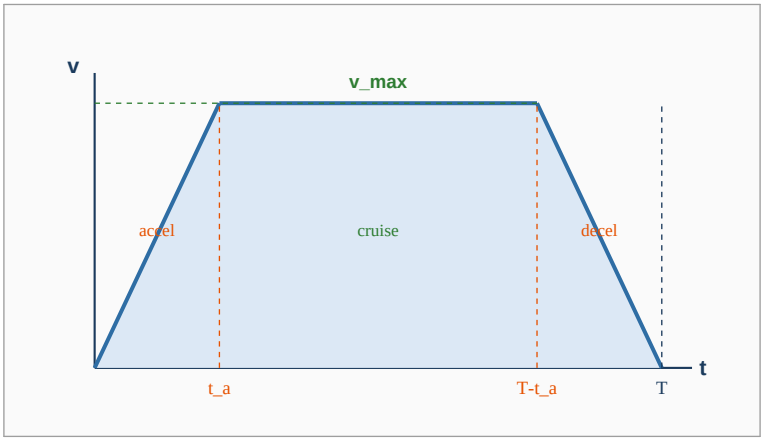


Figure 3 — Trapezoidal velocity profile. Triangle profile (no cruise) used for short segments.

5.1 Profile equations

Acceleration phase duration and distance:

```
t_a = v_max / a_max
d_a = 0.5 * a_max * t_a²
```

If 2·d_a ≥ distance → triangle profile (peak speed limited):

```
t_a    = sqrt(distance / a_max)
v_peak = a_max * t_a
T      = 2 · t_a
```

Otherwise → trapezoidal profile:

```
v_peak   = v_max
t_cruise = (distance - 2·d_a) / v_max
T        = 2·t_a + t_cruise
```

Cumulative arc-length at time *t*:

```
if t ≤ t_a              :  s = 0.5 · a_max · t²
if t_a < t ≤ T - t_a    :  s = d_a + v_peak · (t - t_a)
if t > T - t_a          :  s = distance - 0.5 · a_max · (T - t)²
```

5.2 Speed presets (from blind_pick_place.py)

Preset	v_max (mm/s)	a_max (mm/s²)	Motor ω (rad/s)	Motor α (rad/s²)
low	80	200	1.0	2.0
medium	2000	5000	25.0	50.0
max	4000	10000	50.0	100.0

The Simulink model defaults to the 'low' preset (v=80 mm/s, a=200 mm/s²). Change the constants in the entry actions of any motion state to switch presets.

6 Delta Robot Kinematics

6.1 Geometry constants

Symbol	Value (mm)	Meaning
e	35	End-effector equilateral triangle side
f	157	Base equilateral triangle side
re	400	Forearm (distal link) length
rf	200	Upper arm (proximal link) length

6.2 Inverse kinematics (xyz → θ■, θ■, θ■)

Each arm angle is solved in its own YZ-plane after rotating the XY plane by 0°, –120°, +120° for arms 1, 2, 3. The sub-problem for arm 1 (others identical after rotation):

```
y1 = -0.5 · tan(30°) · f
y0 ← y0 - 0.5 · tan(30°) · e

a  = (x0² + y0² + z0² + rf² - re² - y1²) / (2·z0)
b  = (y1 - y0) / z0
d  = -(a + b·y1)² + rf·(b²·rf + rf)     ← discriminant

yj = (y1 - a·b - sqrt(d)) / (b² + 1)
zj = a + b·yj
θ  = atan2d(-zj, y1 - yj)               ← degrees
```

If *d* < 0 the point is unreachable for that arm → status = –1. All three arms must return status = 0 and θ > 0 for the solution to be valid.

6.3 Workspace limits (from config.py)

Limit	Value	Note
X_LIMIT	±151.563 mm	Bounding box half-width in X
Y_LIMIT	±151.563 mm	Bounding box half-width in Y
Z_MIN	–500 mm	Maximum depth
Z_MAX	–323 mm	Ceiling (IK fails above this)

The bounding box is a rectangular over-approximation. check_ws() also runs IK to reject corners of the box that are outside the reachable sphere.

7 Simulink / Stateflow Implementation

7.1 Chart configuration

Setting	Value	Reason
ActionLanguage	MATLAB	Allows full MATLAB syntax in state actions
StateMachineType	Classic	Moore machine — outputs set on entry
ChartUpdate	DISCRETE	Runs at fixed sample time, no continuous solver
SampleTime	0.05 s (20 Hz)	Matches trajectory dt — one waypoint per step

7.2 Chart data

Name	Type	Scope	Description
trigger	double	Input	1 = trigger active (rising edge detected internally)
pick_x/y/z	double	Input	Pick position in mm
place_x/y/z	double	Input	Place position in mm
sim_t	double	Input	Simulation time from Clock block
state_id	double	Output	Current FSM state (0–5)
theta1/2/3_out	double	Output	Motor angle commands in radians
gripper_out	double	Output	1 = gripper closed, 0 = open
traj [3000×3]	double	Local	Pre-built waypoint buffer (x, y, z columns)
wp_idx	double	Local	Current waypoint index into traj
wp_count	double	Local	Total valid waypoints in traj
settle_timer	double	Local	Absolute time when gripper dwell ends
last_x/y/z	double	Local	Last commanded Cartesian position (for RESETTING)
busy	double	Local	1 while a cycle is running (blocks re-trigger)
traj_done	double	Local	1 when all waypoints in traj have been sent
traj_failed	double	Local	1 if any IK call returns status ≠ 0

7.3 Embedded MATLAB functions (Stateflow.EMFunction)

Function	Signature	Maps to (Python)
angle_yz	[status,θ] = angle_yz(x,y,z,e,f,re,rf)	_delta_calcAngleYZ
delta_ik	[status,t1,t2,t3] = delta_ik(x,y,z)	delta_calcInverse
check_ws	ok = check_ws(x,y,z)	check_workspace
gen_traj_seg	[wps,n] = gen_traj_seg(p0,p1,v,a,dt)	linear_waypoints

All four functions live at chart scope. delta_ik calls angle_yz — Stateflow resolves intra-chart function calls during simulation.

7.4 How a motion state executes a trajectory

Every motion state (MOVING_TO_PICK, MOVING_TO_PLACE, RESETTING) uses the same two-phase pattern:

```
% ■■ entry: build the full multi-segment path once ■■■■■■■■■■■■■■■■■■
wp_count = 0;  wp_idx = 1;
[sg, n] = gen_traj_seg(x0,y0,z0, x1,y1,z1, 80, 200, 0.05);
traj(1:n, :) = sg(1:n, :);  wp_count = n;
% ... repeat for each subsequent segment ...

% ■■ do: step one waypoint per 20 Hz tick ■■■■■■■■■■■■■■■■■■■■■■■■■■■
if wp_idx <= wp_count
    [st, t1, t2, t3] = delta_ik(traj(wp_idx,1), traj(wp_idx,2), traj(wp_idx,3));
    if st == 0
        theta1_out = t1 * pi/180;
        theta2_out = t2 * pi/180;
        theta3_out = t3 * pi/180;
    else
        traj_failed = 1;   % → error transition fires next tick
    end
    last_x = traj(wp_idx,1);  last_y = traj(wp_idx,2);  last_z = traj(wp_idx,3);
    wp_idx = wp_idx + 1;
else
    traj_done = 1;   % → normal transition fires next tick
end
```

8 How to Run

8.1 Generate the .slx model

```
>> cd /path/to/delta_ws
>> run('delta_blind_pick_place_sim.m')
% Output: delta_blind_pick_place.slx
```

8.2 Simulate

```
>> sim('delta_blind_pick_place')
% or press the Run button in Simulink
```

8.3 Change pick / place positions

Double-click any of the six `Constant` source blocks (`pick_x` … `place_z`) and enter the desired value in mm. All positions are validated against the workspace limits on every IK call — an out-of-reach point sets `traj_failed` and triggers the error recovery path.

8.4 Change speed

In each motion state's **entry** action, edit the three constants:

```
V = 80;    % mm/s   — max Cartesian speed
A = 200;   % mm/s²  — max Cartesian acceleration
DT = 0.05; % s      — waypoint interval (keep = chart SampleTime)
```

Use the presets from Section 5.2. Increasing speed reduces cycle time but requires matching motor velocity/acceleration limits in `DeltaMotorController`.

9 ROS 2 ↔ Simulink Mapping

ROS 2 concept	Simulink equivalent
BlindPickAndPlace ROS 2 node	BlindPickPlace_FSM Stateflow chart
/delta/trigger_pick service	Trigger Pulse Generator (rising edge)
/delta/blind_target topic	pick_x/y/z Constant blocks
/delta/robot_state publisher	state_id output (0–5 integer)
DeltaMotorController.execute_trajectory()	do: action — IK per tick → theta_out
linear_waypoints() in trajectory.py	gen_traj_seg() EMFunction in chart
delta_calcInverse() in fk_ik.py	delta_ik() EMFunction in chart
check_workspace() in fk_ik.py	check_ws() EMFunction in chart
PneumaticGripper.close() / open()	gripper_out = 1 / 0 in entry action
close_settle_s / open_settle_s (0.5 s)	settle_timer = sim_t + 0.5
SAFE_TRANSIT_Z = –400 mm	TZ = min([pick_z, place_z, –400]) in entry
ENABLE_MOTORS flag	Not needed — simulation always 'runs'
threading.Thread (daemon)	Stateflow do: action (runs each tick)